

App Dev-I Project Report

1. Student Details

Name: Aniruddha Shukla

Roll Number: 23f2001083

Email: 23f2001083@ds.study.iitm.ac.in

About Me: I am a dual-degree student pursuing a Bachelor of Technology in CSE at Kamla Nehru Institute of Technology, Sultanpur, along with a Bachelor of Science in Data Science and Applications from IIT Madras. I am interested in building solutions for real world problems and have experience in web application development, Machine Learning and Artificial Intelligence.

2. Project Details

Project Title: Trekking Management Application

Problem Statement:

Build a **Trekking Management Application** web application that allows **Admin, Trek Staff, and Users (Trekkers)** to interact with the system based on their roles.

Approach: The application was developed by first designing the database schema, followed by authentication, role-based access control, CRUD operations, backend validations, Bootstrap-based interfaces, and optional enhancements such as REST APIs and dashboard visualizations.

3. AI/LLM Declaration

ChatGPT (GPT-5.5) was used to assist with API documentation, chart visualization, Bootstrap styling suggestions, JavaScript logic debugging, role-based login decorator function, and few validation suggestions.

Overall, the extent of AI/LLM usage is around 30%, limited to code suggestion and tips only.

All final implementation of logic, debugging and integration was done manually.

4. Technologies and Frameworks Used

- **Flask:** Backend Framework
 - **Flask-Login:** Authentication and Session Management
 - **SQLAlchemy:** Object Relational Mapper for SQLite Database
 - **SQLite:** Database for storing data
 - **Jinja2:** Server-side templates
 - **Bootstrap 5:** Responsive and clean UI
 - **HTML5:** Frontend
 - **CSS:** Styling
 - **JavaScript:** Client-side interaction and chart integration
 - **Werkzeug:** Password Hashing and file uploading
 - **Python:** Backend Programming Language
 - **Chart.js:** For creating Charts and Visualizations
-

5. Database Schema and ER Diagram

Models:

User: The User table stores information related to all the registered users in the system including administrator, Trekking Staff and the Trekkers(users)

Trek: The Trek table stores complete information about trekking events managed by the administrator.

Booking: The Booking table records every trek booking made by users.

Staff Profile: The staff profile table stores additional information specific to Trek Staff members.

Staff Notification: The notification table stores notifications generated whenever trek assignments or bookings change.

The Database relationships are summarized below:

User → Booking: One-to-Many

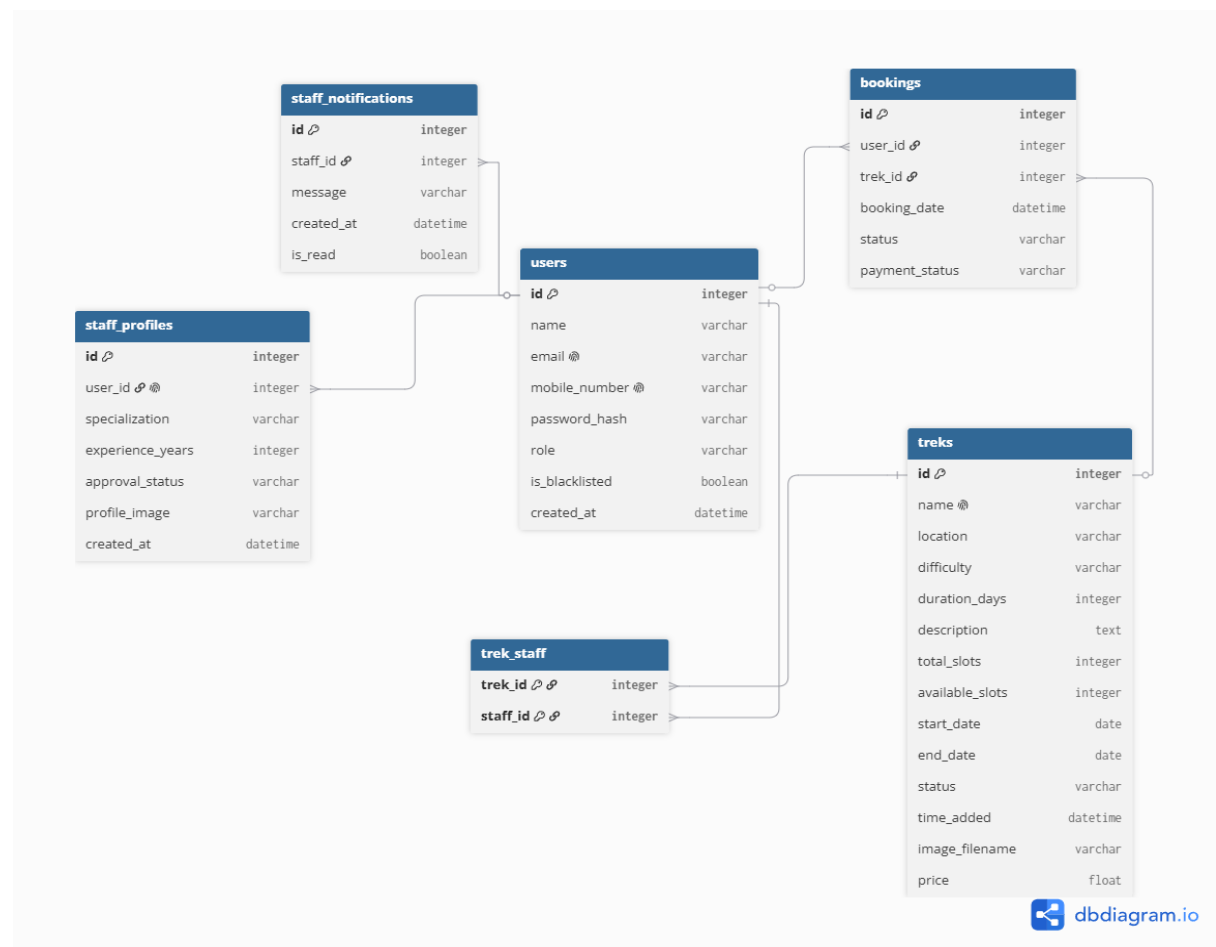
Trek → Booking: One-to-Many

Staff(User) ↔ Trek: Many-to-Many (staff assignment through trek_staff)

User → Staff Profile: One-to-One

User → Staff Notification: One-to-Many

ER Diagram (dbdiagram.io was used to create the diagram)



6. API Resource Endpoints

JSON based REST APIs were implemented using a separate Flask Blueprint (api.py) with the URL prefix /api. The APIs provide access to trek details, user details, booking details, trek participation, staff approval and booking cancellation. The endpoints use appropriate HTTP methods (GET, POST, and DELETE) and return responses in JSON format with suitable HTTP status codes. Access to the APIs is secured using Flask-Login authentication and role-based authorization decorators to ensure that only authorized users can access or modify data.

Endpoint	Method	Description
/api/treks/<id>	GET	Retrieve trek details
/api/users/<id>	GET	Retrieve user details
/api/bookings/<id>	GET	Retrieve booking details
/api/treks/<id>/participants	GET	Retrieve trek participants
/api/staff/<id>/approve	POST	Approve a staff application
/api/bookings/delete/<id>	DELETE	Cancel a booking

7. Architecture And Features

- run.py: flask backend starting script
- app folder:
 - main_routes.py: contains route open to all
 - /routes: contains routes files specific to authentication, admin, staff and user
 - /templates: jinja2 HTML frontend templates
 - /static: CSS, JS, images and uploaded images are stored here
 - models.py: database models using SQLAlchemy
- __init_db__.py: Database initialization script
- Utils folder: contains helper decorators and application constants
- README.md: Contains project setup and execution instructions

Implemented Features

Administrator Features:

- Admin Dashboard
- Create, edit and delete treks
- Approve or reject staff applications
- Assign staff to treks
- Manage users and staff
- Blacklist and Whitelist users
- View booking history
- View trek participants

Trek Staff Features:

- Staff Dashboard
- View assigned Treks
- Update Trek status
- Update total slots

- Manage participants
- Complete Treks
- Receive assigned trek related notifications
- Update profile

Trekker Features:

- Register and login
- Browse available treks
- Search and filter treks
- Book treks
- Cancel bookings
- View active bookings
- View trek history
- Apply for staff role
- Update profile

Business Rules Implemented:

The application enforces several business rules to maintain consistency and prevent invalid operations. Some of which are:

- Only Open Treks can be booked
 - Duplicate Trek booking is not allowed
 - Overbooking beyond available slots not allowed
 - User cannot cancel booking after trek starts
 - Only assigned staff member can manage a trek
 - Completed Treks cannot be modified
 - Treks cannot be opened without assigning at least one staff member
 - Staff assignment overlap is checked to avoid conflicts
-

8. Image Credits

The images used in the landing page are sourced from publicly available websites for educational purposes.

Hero.jpg: [Hero image link](#)

featured-bg.jpg: [featured-bg image link](#)

9. Video Walkthrough Link

Google Drive: [Project Video](#)